

Simulations Project

Computer Animation - MIRI

Pau Adell (pau.adell@estudiantat.upc.edu)

November 2025

1 L0: Integrators

In the next part of the laboratory, we implement different types of integrators (numerical solvers) to approximate the solution of differential equations and analyze their performance and accuracy.

Symplectic Euler

We started with the given simplest solver (euler code):

$$X_{t+\Delta t} = X_t + \Delta t \cdot X'_t$$

Midpoint

We implemented a second-order accuracy method, achieving more precise results (midpoint code):

$$X_{t+\Delta t} = X_t + \Delta t \cdot X'_{t+\Delta t/2}$$

RK2

We also implemented Runge–Kutta order 2 method (RK2 - code):

$$\begin{aligned}k_1 &= f(t, X_t) \\k_2 &= f(t + \Delta t, X_t + \Delta t \cdot k_1) \\X_{t+\Delta t} &= X_t + \frac{\Delta t}{2}(k_1 + k_2)\end{aligned}$$

RK4

And its order 4 method as well (RK4 - code):

$$\begin{aligned}k_1 &= f(t, X_t) \\k_2 &= f\left(t + \frac{\Delta t}{2}, X_t + \frac{\Delta t}{2} \cdot k_1\right) \\k_3 &= f\left(t + \frac{\Delta t}{2}, X_t + \frac{\Delta t}{2} \cdot k_2\right)\end{aligned}$$

$$k_4 = f(t + \Delta t, X_t + \Delta t \cdot k_3)$$

$$X_{t+\Delta t} = X_t + \frac{\Delta t}{6}(k_1 + 2 \cdot k_2 + 2 \cdot k_3 + k_4)$$

Simplectic Euler

A first implementation that conserves the structure of phase space (simplectic euler code):

$$V_{t+\Delta t} = V_t + \Delta t \cdot \frac{F_t}{m}$$

$$P_{t+\Delta t} = P_t + \Delta t \cdot V_{t+\Delta t}$$

Verlet

Our final integrator implementation to test which consists on energy conservation and stability(verlet code):

$$P_{t+\Delta t} = P_t + k \cdot (P_t - P_{t-\Delta t}) + \frac{\Delta t^2}{m} F$$

$$V_{t+\Delta t} = \frac{P_{t+\Delta t} - P_t}{\Delta t}$$

And this is the table of the results we get for each of our integrators implemented:

	Analytic	Euler	Midpoint	RK2	RK4	Symplectic Euler	Verlet
X overlap	2.51785	24.60134	2.09627	2.10091	2.51885	1.05987	1.89816
V overlap	-1.62644	12.4564	-4.35794	-4.35892	-1.56072	2.65767	-1.56679

Table 1: Overlap comparison for different integration methods

There were not many difficulties in order to implement this section, apart from understanding the equations and understanding each step. One thing to take into consideration is that for the Verlet integrator an additional variable had to be added to know the first step (and not use a non-existent previous value), meaning that now we had to update all scenes with reset buttons in order to update its value. It probably took around 4-6h.

2 L1: Forces

Projectiles Scene

The first thing implemented here was the floor collision test. We test if from the previous position to the current one (after applying the integration step) the position is lower than the floor. In that case we compute the direction from one step to the other and compute where it should hit the ground. We set the new computed position and stop the system like in the image:

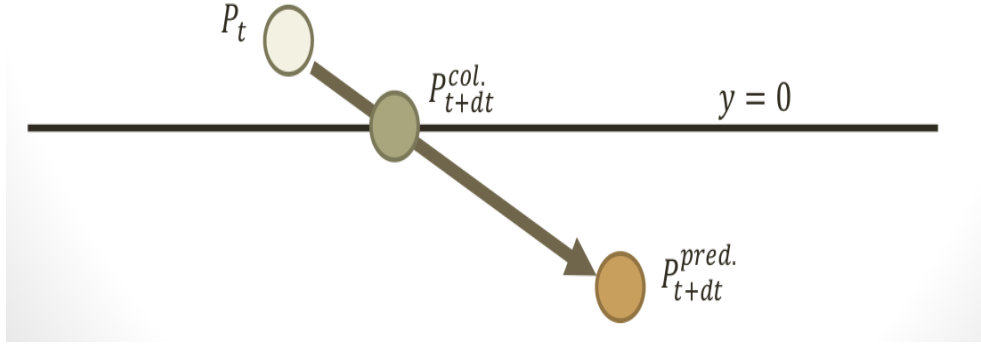


Figure 1: Floor collision detection and response.

Elastic Rope Scene

For this scene we had to implement the force between two particles and we did like specified (ForceSpring - code):

$$F_1^{m1}(P_1, P_2) = \left(k_e \cdot (\|P_2 - P_1\| - L_{12}) + k_d \cdot (v_2 - v_1) \cdot \frac{P_2 - P_1}{\|P_2 - P_1\|} \right) \cdot \frac{P_2 - P_1}{\|P_2 - P_1\|} \quad (1)$$

$$F_2^{m1}(P_1, P_2) = -F_1^{m1} \quad (2)$$

N-Body Scene

For the N-body problem scene, apart from adding it, we had to implement the following gravitational effect between particles as a new force (ForceGravitation - code):

$$\mathbf{F} = \frac{G \cdot m_i \cdot m_j}{d(\mathbf{P}_i, \mathbf{P}_j)^2} \cdot \frac{\mathbf{P}_i - \mathbf{P}_j}{\|\mathbf{P}_i - \mathbf{P}_j\|} \quad (3)$$

Here the only difficulties as mentioned before were that if Verlet not implemented correctly, the velocity and direction of the particles would not work properly as we did not have an appropriate previous step. Apart from it we had no issues with it and cannot recall how much time it took to implement. As comments, I always disliked how easily it is to mess up adding a new scene, button, or connecting anything using QT but that is just subjective preference.

3 L2: Particle system and collisions

For all the next cases, we implemented two functions. First function to know if the particle is inside the other object, and the second one computes the continuous collision detection and stores the information of where it would hit and what would be the normal (as if the collision position hit a plane).

Plane collision

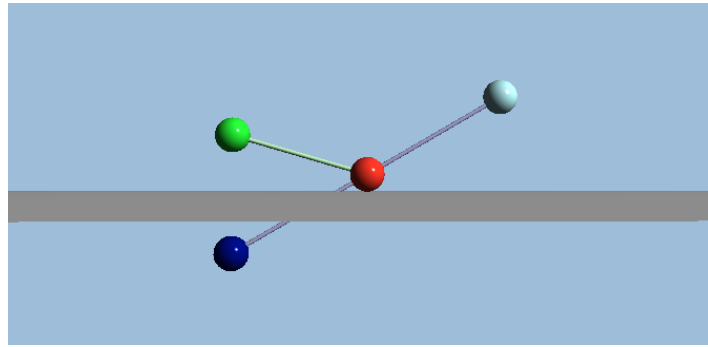


Figure 2: Particle continuous collision detection with a plane.

Sphere collision

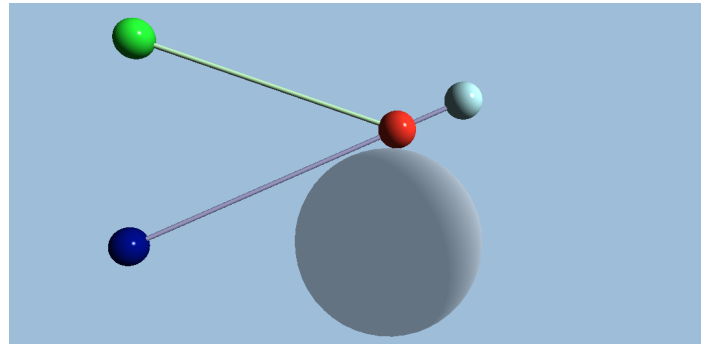


Figure 3: Particle continuous collision detection with a sphere.

AABB collision

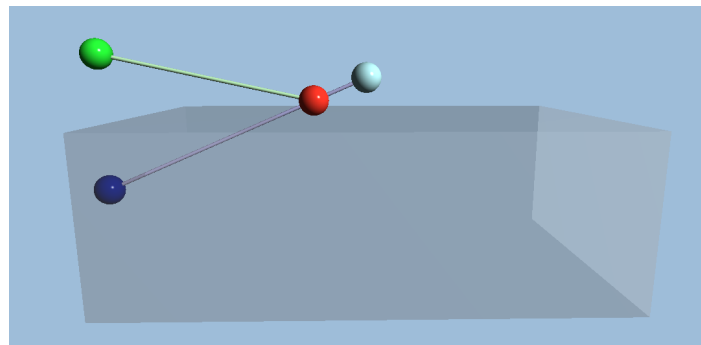


Figure 4: Particle continuous collision detection with an AABB.

Resolve collision

And once those functions are implemented, we can compute the response as a function following the next criteria (CollisionResponse - code):

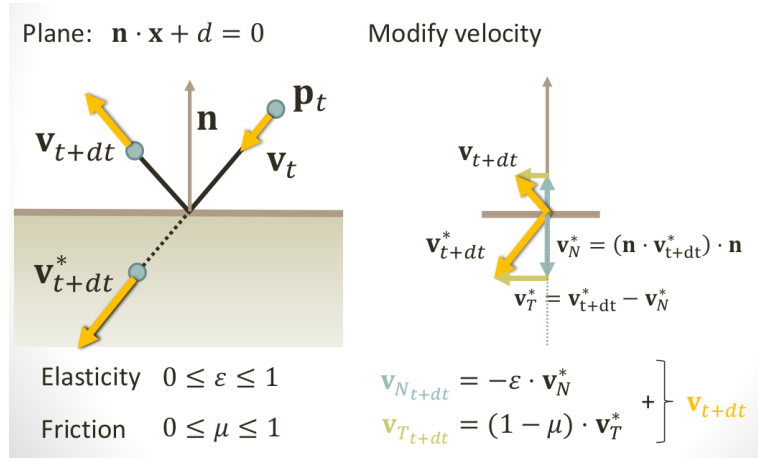


Figure 5: Expected position and velocities after collision.

For this part of course, each collision had its own challenges; some of them, for example, at the time of computing the position of the collision, sometimes you would compute the point where the particle would hit the surface but that was not the center of where the particle would be at collision time. Thanks to the scene where we can visualize this, these slight errors could be seen and solved to be as precise as possible, so its very useful! This part took around 8 hours.

4 L3: Provot's cloth simulation

This is an image to represent the provot layout that we implemented and to show the springs that we added to each particle.

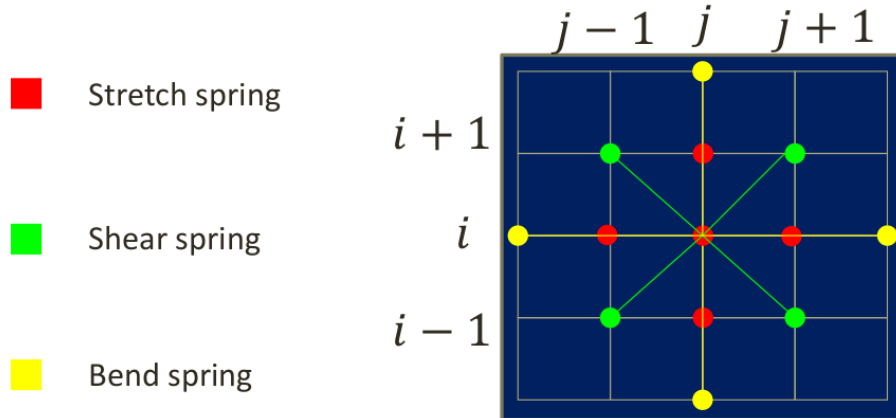


Figure 6: PROVOT's layout springs.

One thing to take into consideration at the time of implementation is to make sure that if you iterate through all the particles you don't want duplicated springs. This was the code that we used to add all the springs equally to the image and not repeating any (ProvotLayout - code)

We also added some tilt applied to the z direction because we added a sphere to test the collisions.

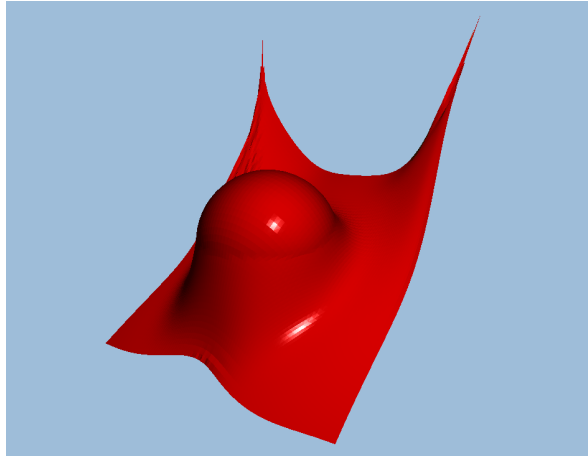


Figure 7: Visual representation of cloth colliding with sphere.

But we have to mention a very important issue we experimented with and that we did not have enough time to solve due to time constraints. For all the integration methods but the basic euler, my fixed particles would not stay fixed. I must say that after looking it for a while, it seems like the problem lies in the update function where I modify the position velocities and forces of each particle. At some point after setting the fixed particles with 0 force and velocity I might update them after via stretching from other particles or due to gravity. I have to admit I spend a lot of time here as I tried changing the mass of those particles to 0 and few other ways to solve the problem which might have a very simple solution but I wasn't fortunate enough to find it with the time I dedicated. I should have spent more time, and this also made me rush to implement the next SPH lab.

5 L4: SPH

In the following scene, to create a particle simulator, we made a transparent cube where particles can be spawned inside — either several at once or gradually until reaching a set limit. We implemented these features in the UI, allowing the user to adjust both the number of particles that can appear at a time and the maximum total number of particles, as shown in the image below:

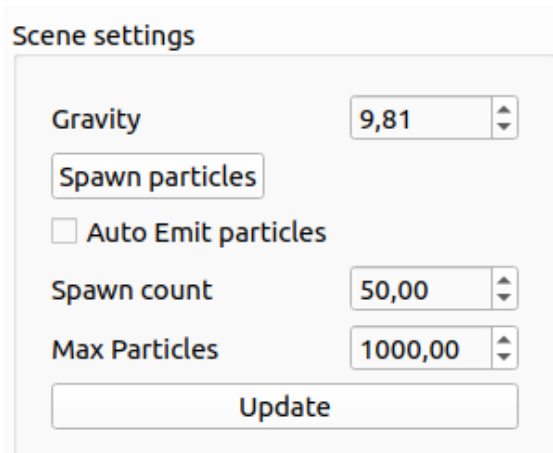


Figure 8: Visual representation of cloth colliding with sphere.

I did not have enough time to implement the hash functions to accelerate particle collisions and for this reason I had to assign SPH parameters that would make them separate more from each other and like this I could still visualize and experiment that they actually behave like a liquid even having less particles:

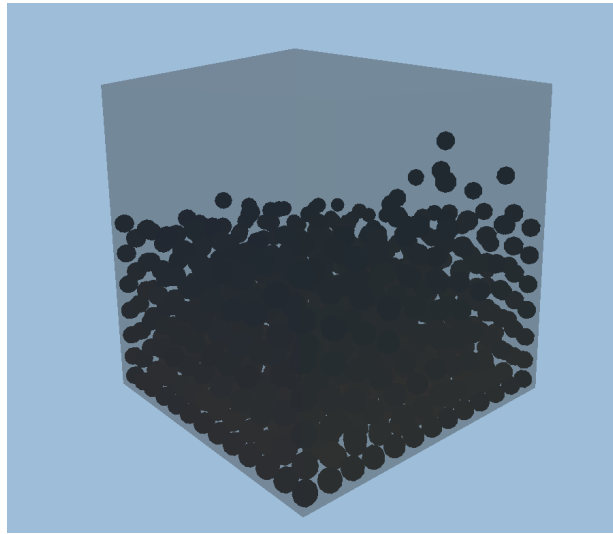


Figure 9: Visual representation of cloth colliding with sphere.

To do so, what I implemented is with the right mouse button the ability to move the cube and see how all the particles move inside as if they were a fluid and it looked very accomplished.

6 Annex

Euler solver:

```

1 void IntegratorEuler::step(ParticleSystem &system, double dt) {
2     double t0 = system.getTime();
3     Vecd x0 = system.getState();
4     Vecd dx = system.getDerivative();
5     Vecd x1 = x0+dt*dx;
6     system.setState(x1);
7     system.setTime(t0+dt);
8     system.updateForces();
9 }

```

Midpoint solver:

```

1 void IntegratorMidpoint::step(ParticleSystem &system, double dt) {
2     const double t0 = system.getTime();
3     const Vecd x0 = system.getState();
4
5     // Compute initial Euler step
6     system.setState(x0);
7     system.setTime(t0);
8     system.updateForces();
9     const Vecd k1 = system.getDerivative();
10
11    // Midpoint state (and time)
12    const double tm = t0 + 0.5*dt;
13    const Vecd xm = x0 + 0.5*dt * k1;
14
15    // k2 at midpoint
16    system.setState(xm);
17    system.setTime(tm);
18    system.updateForces();
19    const Vecd k2 = system.getDerivative();
20
21    // Full step using k2
22    const Vecd x1 = x0 + dt * k2;
23
24    system.setState(x1);
25    system.setTime(t0 + dt);
26    system.updateForces();
27 }

```

RK2 solver:

```

1 void IntegratorRK2::step(ParticleSystem &system, double dt) {
2     const double t0 = system.getTime();
3     const Vecd x0 = system.getState();
4
5     // k1 at (t0, x0)
6     system.setState(x0);
7     system.setTime(t0);
8     system.updateForces();

```

```

9     const Vecd k1 = system.getDerivative();
10
11     // Euler predictor to t0+dt
12     const Vecd xE = x0 + dt * k1;
13
14     // k2 at (t0+dt, xE)
15     system.setState(xE);
16     system.setTime(t0 + dt);
17     system.updateForces();
18     const Vecd k2 = system.getDerivative();
19
20     // Average the slopes
21     const Vecd x1 = x0 + 0.5*dt * (k1 + k2);
22
23     system.setState(x1);
24     system.setTime(t0 + dt);
25     system.updateForces();
26 }

```

RK4 solver:

```

1 void IntegratorRK4::step(ParticleSystem &system, double dt) {
2     const double t0 = system.getTime();
3     const Vecd x0 = system.getState();
4
5     // k1 at (t0, x0)
6     system.setState(x0);
7     system.setTime(t0);
8     system.updateForces();
9     const Vecd k1 = system.getDerivative();
10
11     // k2 = f(t0 + dt/2, x0 + dt/2 * k1)
12     const double th = t0 + 0.5*dt;
13     Vecd xtmp = x0 + 0.5*dt * k1;
14     system.setState(xtmp);
15     system.setTime(th);
16     system.updateForces();
17     const Vecd k2 = system.getDerivative();
18
19     // k3 = f(t0 + dt/2, x0 + dt/2 * k2)
20     xtmp = x0 + 0.5*dt * k2;
21     system.setState(xtmp);
22     system.setTime(th);
23     system.updateForces();
24     const Vecd k3 = system.getDerivative();
25
26     // k4 = f(t0 + dt, x0 + dt * k3)
27     xtmp = x0 + dt * k3;
28     system.setState(xtmp);
29     system.setTime(t0 + dt);
30     system.updateForces();
31     const Vecd k4 = system.getDerivative();
32
33     // advance:  $x_{n+1} = x_n + dt/6 (k1 + 2k2 + 2k3 + k4)$ 

```

```

34     const Vecd x1 = x0 + (dt/6.0) * (k1 + 2.0*k2 + 2.0*k3 + k4);
35
36     system.setState(x1);
37     system.setTime(t0 + dt);
38     system.updateForces();
39 }

```

Simplectic Euler solver:

```

1
2 void IntegratorSymplecticEuler::step(ParticleSystem &system, double dt) {
3     double t0 = system.getTime();
4
5     // v_{t+dt} = v_t + dt * a_t
6     Vecd v0 = system.getVelocities();
7     Vecd a0 = system.getAccelerations(); // F/m per particle
8     Vecd v1 = v0 + dt * a0;
9     system.setVelocities(v1);
10
11    // p_{t+dt} = p_t + dt * v_{t+dt}
12    Vecd p0 = system.getPositions();
13    Vecd p1 = p0 + dt * v1;
14    system.setPositions(p1);
15
16    // advance time and refresh forces for the next step
17    system.setTime(t0 + dt);
18    system.updateForces();
19 }

```

Verlet solver:

```

1 void IntegratorVerlet::step(ParticleSystem &system, double dt) {
2     double k = 0.99;
3     const double t0 = system.getTime();
4
5     // Get current and previous positions
6     Vecd p_t = system.getPositions(); // P_t
7     Vecd p_prev = system.getPreviousPositions(); // P_{t - dt}
8
9     // Forces -> accelerations
10    system.setTime(t0);
11    system.updateForces();
12    Vecd a0 = system.getAccelerations();
13
14    // Verlet position update
15    // P_{t+dt} = P_t + k * (P_t - P_{t-dt}) + (dt^2) * (F/m)
16    Vecd p_next = p_t + k * (p_t - p_prev) + (dt * dt) * a0;
17
18    //Optional: compute velocity approximation (for visualization/debug)
19    Vecd v_next = (p_next - p_t) / dt;
20
21    // Update system state
22    system.setPreviousPositions(p_t); // Shift: current becomes previous
23    system.setPositions(p_next);

```

```

24     system.setVelocities(v_next);
25     system.setTime(t0 + dt);
26     system.updateForces();
27 }

```

ForceSpring function:

```

1 void ForceSpring::apply()
2 {
3     // we need 2 particles
4     if (particles.size() < 2) return;
5
6     Particle* p1 = getParticle1();
7     Particle* p2 = getParticle2();
8
9     // current spring vector
10    // P2 - P1
11    Vec3 d = p2->pos - p1->pos;
12    // ||P2 - P1||
13    double len = d.norm();
14    if (len < 1e-9) return;           // avoid division by zero
15
16    // unit direction from p1 to p2
17    Vec3 n = d / len;
18
19    // rest length from the spring object (you have getRestLength())
20    double L0 = getRestLength();
21
22    // relative velocity along spring
23    Vec3 relV = p2->vel - p1->vel;
24    // (v2 - v1) * n
25    double relVAlong = relV.dot(n);
26
27    // slide formula:
28    // ( ke * (|p2 - p1| - L0) + kd * (v2 - v1)n ) * n
29    double fMag = ks * (len - L0) + kd * relVAlong;
30    Vec3 f = fMag * n;
31
32    // apply equal and opposite forces
33    p1->force += f;
34    p2->force -= f;
35 }

```

ForceGravitation function:

```

1 void ForceGravitation::apply() {
2     const size_t n = particles.size();
3     if (n < 1) return;
4
5     const double eps2 = 1e-12;    // avoid singularities / zero distance
6
7     if (attractor) {
8         const Eigen::Vector3d xa = attractor->pos;
9         const double ma = attractor->mass;

```

```

10
11     for (size_t i = 0; i < n; ++i) {
12         Particle* pi = particles[i];
13         if (pi == attractor) continue;           // just in case
14
15         // Pi-Pj
16         Eigen::Vector3d r = xa - pi->pos;         // toward attractor
17         const double r2_raw = r.squaredNorm();
18         if (r2_raw < eps2) continue;
19
20         const Eigen::Vector3d dir = r / std::sqrt(r2_raw); // (Pi-Pj)/||Pi-Pj||
21         const double denom = a + b * r2_raw;         // smoothing
22         const double mag = (G * ma * pi->mass) / denom; // G*mi*mj/d^2 (
softened)
23
24         pi->force += mag * dir;
25     }
26     return;
27 }
28
29 for (size_t i = 0; i < n; ++i) {
30     Particle* pi = particles[i];
31     for (size_t j = i + 1; j < n; ++j) {
32         Particle* pj = particles[j];
33
34         // Pi-Pj
35         Eigen::Vector3d r = pi->pos - pj->pos;
36         const double r2_raw = r.squaredNorm();
37         if (r2_raw < eps2) continue;
38
39         const Eigen::Vector3d dir = r / std::sqrt(r2_raw); // (Pi-Pj)/||Pi-
Pj||
40         const double denom = a + b * r2_raw;         // softened d^2
41         const double mag = (G * pi->mass * pj->mass) / denom; // G*mi*mj/d^2 (
softened)
42
43         const Eigen::Vector3d F = mag * dir;
44         pi->force += F;
45         pj->force -= F;                               // Newton 3rd
law
46     }
47 }
48 }

```

Collision resolve function:

```

1 void Collider::resolveCollision(Particle* p, const Collision& col, double kElastic,
   double kFriction) const
2 {
3     Eigen::Vector3d n = col.normal.normalized();
4     double h = dot(n, p->pos - col.position);
5
6     // Only resolve if the particle is penetrating the plane
7     if (h < kEps)

```

```

8      {
9          // --- 1. Correct position (push out of plane based on kElastic)
10         p->pos -= (1.0 + kElastic) * h * n;
11
12         // --- 2. Decompose velocity
13         Eigen::Vector3d v = p->vel;
14         Eigen::Vector3d vN = dot(v, n) * n;           // Normal component
15         Eigen::Vector3d vT = v - vN;                 // Tangential component
16
17         // --- 3. Modify velocity
18         //          VNt+dt      +          VTt+dt
19         p->vel = -kElastic * vN + (1.0 - kFriction) * vT;
20     }
21 }

```

Implementation of PROVOT layout springs:

```

1  auto addSpring = [&](int i, int j,
2                      int ni, int nj,
3                      std::vector<ForceSpring*>& bag)
4  {
5      // bounds
6      if (ni < 0 || nj < 0 || ni >= numParticlesX || nj >= numParticlesY)
7          return;
8
9      int idxA = i * numParticlesY + j;
10     int idxB = ni * numParticlesY + nj;
11
12     Particle* a = system.getParticle(idxA);
13     Particle* b = system.getParticle(idxB);
14
15     double L0 = (a->pos - b->pos).norm();
16
17     // ks, kd = 0 for now; we set real values with updateSprings()
18     ForceSpring* s = new ForceSpring(a, b, L0, 0.0, 0.0);
19     bag.push_back(s);
20     system.addForce(s);
21 };
22
23 // optional, just to avoid reallocs
24 springsStretch.reserve(numParticlesX * numParticlesY * 2);
25 springsShear.reserve(numParticlesX * numParticlesY * 2);
26 springsBend.reserve(numParticlesX * numParticlesY * 2);
27
28 for (int i = 0; i < numParticlesX; ++i) {
29     for (int j = 0; j < numParticlesY; ++j) {
30
31         // -----
32         // 1) STRETCH springs (structural)
33         //    only to +x (right) and +y (up) to avoid dups
34         // -----
35         addSpring(i, j, i + 1, j,      springsStretch); // right (i+1, j)
36         addSpring(i, j, i,      j + 1, springsStretch); // up    (i, j+1)
37

```

```
38         // -----
39         // 2) SHEAR springs (diagonals)
40         //     again, only "forward" diagonals
41         // -----
42         addSpring(i, j, i + 1, j + 1, springsShear);    // up-right
43         addSpring(i, j, i + 1, j - 1, springsShear);    // down-right (
addSpring checks bounds)
44
45         // -----
46         // 3) BEND springs (2 cells away)
47         //     only forward to avoid dups
48         // -----
49         addSpring(i, j, i + 2, j,    springsBend);      // 2 to the right
50         addSpring(i, j, i,    j + 2, springsBend);      // 2 up
51     }
52 }
```